Θωράκιση Διακομιστή MQTT Ενός Συστήματος IOT

9^{H} ΕΡΓΑΣΙΑ

Κωνσταντίνος Μυλωνάς | mli25036 | 03.2026

Πίνακας περιεχομένων

Εισαγωγή 3
Ερώτημα 1: Αυθεντικοποίηση Clients με Username/Password 3
Περιγραφή 4
Βήματα Υλοποίησης 4
Ερώτημα 2: Έλεγχος Πρόσβασης με ACL 6
Περιγραφή 6
Βήματα Υλοποίησης 6
Ερώτημα 3: Ασφαλής Επικοινωνία με TLS/Certificates 8
Περιγραφή 8
Βήματα Υλοποίησης 8
Ερώτημα 4: Single Point of Failure & Αντιμετώπιση DDoS 12
Περιγραφή 12
i. Αύξηση Διαθεσιμότητας 12
ii. Περιορισμός DDoS Επιθέσεων 12
Συμπέρασμα 13

Εισαγωγή

Σε ένα σύστημα Internet of Things (IoT), ο MQTT broker αποτελεί τον κεντρικό κόμβο επικοινωνίας μεταξύ των συσκευών. Το πρωτόκολλο MQTT (Message Queuing Telemetry Transport) βασίζεται στο μοντέλο Publish/Subscribe, σύμφωνα με το οποίο οι συσκευές δεν επικοινωνούν απευθείας αλλά μέσω του broker, ο οποίος διαχειρίζεται τις συνδέσεις και την προώθηση μηνυμάτων.

Από προεπιλογή, ο MQTT broker Eclipse Mosquitto δεν είναι ασφαλής — επιτρέπει ανώνυμη πρόσβαση και μη κρυπτογραφημένη επικοινωνία, καθιστώντας τον ευάλωτο σε κυβερνοεπιθέσεις. Η παρούσα εργασία πραγματεύεται τη σταδιακή θωράκισή του μέσω τεσσάρων ερωτημάτων: αυθεντικοποίηση με username/password, έλεγχος πρόσβασης με ACL, κρυπτογράφηση επικοινωνίας με TLS/SSL και αντιμετώπιση Single Point of Failure και DDoS επιθέσεων.

Ερώτημα 1: Αυθεντικοποίηση Clients με Username/Password

Περιγραφή

Ρύθμιση του Mosquitto broker ώστε να δέχεται μόνο πιστοποιημένους χρήστες μέσω username και password, απαγορεύοντας ανώνυμη πρόσβαση. Χωρίς έγκυρα credentials ο broker απορρίπτει οποιαδήποτε σύνδεση.

Βήματα Υλοποίησης

Βήμα 1 — Backup mosquitto.conf

Πριν οποιαδήποτε αλλαγή, αντιγράφηκε το αρχικό αρχείο ρυθμίσεων σε ασφαλές σημείο και μετονομάστηκε σε mosquito.conf.backup.

Βήμα 2 — Δημιουργία Password File

Χρησιμοποιήθηκε το εργαλείο mosquitto_passwd.exe για τη δημιουργία των credentials των δύο clients. Δημιουργήθηκαν δύο χρήστες: publisher_user και subscriber_user

Βήμα 3 — Παραμετροποίηση mosquitto.conf

Προστέθηκαν οι παράμετροι allow_anonymous false και password_file για την απαγόρευση ανώνυμης πρόσβασης.

mosquitto.conf

```
listener 1883
allow_anonymous false
password_file C:/Program Files/mosquitto/passwd.txt
```

Βήμα 4 — Εκκίνηση Broker & Αποτελέσματα

Ο publisher συνδέεται επιτυχώς και στέλνει μετρήσεις θερμοκρασίας.

Command Prompt - py q1_au

```
(venv) C:\Users\conmy\Documents\project9>py q1_auth\publisher.py
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 22.1, 'timestamp': '2026-03-18T14:15:49.21
4114+00:00'}
Publisher connected with CLIENT_ID: iot-publisher-3b82580e
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 21.58, 'timestamp': '2026-03-18T14:15:54.2
15169+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 26.5, 'timestamp': '2026-03-18T14:15:59.21
6138+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 29.32, 'timestamp': '2026-03-18T14:16:04.2
17777+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 22.06, 'timestamp': '2026-03-18T14:16:09.2
18988+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 22.82, 'timestamp': '2026-03-18T14:16:14.2
19984+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 26.7, 'timestamp': '2026-03-18T14:16:19.22
1869+00:00'}
```

Ο subscriber λαμβάνει κανονικά τα μηνύματα.

```
Command Prompt - py q1_au
(venv) C:\Users\conmy\Documents\project9>py q1_auth\subscriber.py
Subscriber ON - Client ID: subscriber-asynnb
Connected to MQTT broker successfully
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 26.5, "timestamp": "2026
-03-18T14:15:59.216138+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 29.32, "timestamp": "202
6-03-18T14:16:04.217777+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 22.06, "timestamp": "202
6-03-18T14:16:09.218988+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 22.82, "timestamp": "202
6-03-18T14:16:14.219984+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 26.7, "timestamp": "2026
-03-18T14:16:19.221869+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 23.46, "timestamp": "202
6-03-18T14:16:24.223348+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 22.81, "timestamp": "202
6-03-18T14:16:29.224826+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 24.24, "timestamp": "202
6-03-18T14:16:34.225744+00:00"}
```

Η σύνδεση χωρίς credentials απορρίπτεται από τον broker.

```
Command Prompt

(venv) C:\Users\conmy\Documents\project9>mosquitto_sub -h localhost -t test/topic
Connection error: Connection Refused: not authorised

(venv) C:\Users\conmy\Documents\project9>
```

Ερώτημα 2: Έλεγχος Πρόσβασης με ACL

Περιγραφή

Υλοποίηση μηχανισμού ελέγχου πρόσβασης σε topics μέσω ACL. Ο publisher_user επιτρέπεται να κάνει μόνο publish στο iot/temperature και ο subscriber_user μόνο subscribe στο ίδιο topic.

Βήματα Υλοποίησης

Βήμα 1 — Δημιουργία ACL αρχείου

Δημιουργήθηκε αρχείο acl.txt με τους κανόνες πρόσβασης.

acl.txt

```
user publisher_user
topic write iot/temperature

user subscriber_user
topic read iot/temperature
```

Βήμα 2 — Παραμετροποίηση mosquitto.conf

Προστέθηκε η παράμετρος acl_file στο αρχείο ρυθμίσεων.

mosquitto.conf

```
listener 1883
allow_anonymous false
password_file C:/Program Files/mosquitto/passwd.txt
acl_file C:/Program Files/mosquitto/acl.txt
```

Βήμα 3 — Εκκίνηση Broker & Αποτελέσματα

Εκκίνηση με το νέο configuration.

Publisher στέλνει κανονικά:

Command Prompt - py q2_acl | Command Prompt - py q2_ac

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\conmy>cd C:\Users\conmy\Documents\project9

C:\Users\conmy\Documents\project9>venv\Scripts\activate

(venv) C:\Users\conmy\Documents\project9>py q2_acl\publisher.py
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 21.39, 'timestamp': '2026-03-18T15:04:52.802095+00:00'}
Publisher connected with CLIENT_ID: iot-publisher-c16e0754
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 28.64, 'timestamp': '2026-03-18T15:04:57.803302+00:00'}
```

Subscriber λαμβάνει κανονικά:

```
Command Prompt - py q2_ac    Command Prompt - py q2_acl

C:\Users\conmy>cd C:\Users\conmy\Documents\project9

C:\Users\conmy\Documents\project9>venv\Scripts\activate

(venv) C:\Users\conmy\Documents\project9>py q2_acl\subscriber.py
Subscriber ON - Client ID: subscriber-pqak9m
Connected to MQTT broker successfully
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 21.39, "timestamp": "2026-03-18T15:04:52.802
095+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 28.64, "timestamp": "2026-03-18T15:04:57.803
302+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 27.36, "timestamp": "2026-03-18T15:05:02.804
591+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 23.6, "timestamp": "2026-03-18T15:05:07.8059
83+00:00"}
```

publisher_user δεν λαμβάνει μηνύματα ως subscriber:

```
Command Prompt - mosquitt

C:\Users\conmy>mosquitto_sub -h localhost -t iot/temperature -u publisher_user -P pub_pass123
```

subscriber_user αποτυγχάνει να κάνει publish:

```
Administrator: Command Prompt - mosquitto.exe  -c mosquitto.conf -v
1773845877: Bridge support available.
1773845877: Persistence support available.
1773845877: TLS support available.
1773845877: TLS-PSK support available.
1773845877: Websockets support available.
1773845877: Plugin builtin-security has registered to receive 'basic-auth' events.
1773845877: Plugin builtin-security has registered to receive 'acl-check' events.
1773845877: Opening ipv6 listen socket on port 1883.
1773845877: Opening ipv4 listen socket on port 1883.
1773845877: mosquitto version 2.1.2 running
1773845886: New connection from ::1:59566 on port 1883.
1773845886: New client connected from ::1:59566 as auto-35F78B76-5979-69A5-B182-8EBF6EEF982E (p4, c1, k60, u'publisher_u
ser').
1773845886: No will message specified.
1773845886: Sending CONNACK to auto-35F78B76-5979-69A5-B182-8EBF6EEF982E (0, 0)
1773845886: Received SUBSCRIBE from auto-35F78B76-5979-69A5-B182-8EBF6EEF982E
1773845886:     iot/temperature (QoS 0)
1773845886: auto-35F78B76-5979-69A5-B182-8EBF6EEF982E 0 iot/temperature
1773845886: Sending SUBACK to auto-35F78B76-5979-69A5-B182-8EBF6EEF982E
1773845893: Client auto-35F78B76-5979-69A5-B182-8EBF6EEF982E [::1:59566] disconnected: connection closed by client.
1773845895: New connection from ::1:59575 on port 1883.
1773845895: New client connected from ::1:59575 as auto-9089E180-AC2A-9D57-613A-FF89AEBFDCA8 (p4, c1, k60, u'subscriber_
user').
1773845895: No will message specified.
1773845895: Sending CONNACK to auto-9089E180-AC2A-9D57-613A-FF89AEBFDCA8 (0, 0)
1773845895: Denied PUBLISH from auto-9089E180-AC2A-9D57-613A-FF89AEBFDCA8 (d0, q0, r0, m0, 'iot/temperature', ... (4 byt
es))
1773845895: Received DISCONNECT from auto-9089E180-AC2A-9D57-613A-FF89AEBFDCA8
1773845895: Client auto-9089E180-AC2A-9D57-613A-FF89AEBFDCA8 [::1:59575] disconnected.
```

Ερώτημα 3: Ασφαλής Επικοινωνία με TLS/Certificates

Περιγραφή

Υλοποίηση κρυπτογραφημένης επικοινωνίας SSL/TLS μεταξύ MQTT clients και Mosquitto broker με χρήση πιστοποιητικών που εκδόθηκαν με OpenSSL. Η επικοινωνία μεταφέρθηκε στο port 8883.

Βήματα Υλοποίησης

Βήμα 1 — Δημιουργία CA Certificate

Δημιουργήθηκε Certificate Authority (CA) με τις εντολές:

openssl genrsa -out ca.key 2048

openssl req -new -x509 -days 1826 -key ca.key -out ca.crt

Βήμα 2 — Δημιουργία Server Certificate

Δημιουργήθηκε το κλειδί και το certificate του broker:

openssl genrsa -out server.key 2048

openssl req -new -key server.key -out server.csr

openssl x509 -req -days 1826 -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt

Βήμα 3 — Περιεχόμενο φακέλου certificates

Select Administrator: Command Prompt

```
C:\Program Files\Mosquitto>dir "C:\Users\conmy\Documents\project9\q3_tls\certs"
 Volume in drive C is OS
 Volume Serial Number is 66FC-9223

 Directory of C:\Users\conmy\Documents\project9\q3_tls\certs

18/03/2026  17:28    <DIR>          .
18/03/2026  17:33    <DIR>          ..
18/03/2026  17:26             1.500 ca.crt
18/03/2026  17:26             1.732 ca.key
18/03/2026  17:28                42 ca.srl
18/03/2026  17:28             1.476 server.crt
18/03/2026  17:28             1.134 server.csr
18/03/2026  17:27             1.732 server.key
               6 File(s)          7.616 bytes
               2 Dir(s)  443.466.125.312 bytes free
```

Βήμα 4 — Παραμετροποίηση mosquitto.conf

mosquitto.conf | publisher.py | subscriber.py

```
listener 8883
allow_anonymous false
password_file C:/Program Files/mosquitto/passwd.txt
acl_file C:/Program Files/mosquitto/acl.txt

cafile C:/Program Files/mosquitto/ca.crt
certfile C:/Program Files/mosquitto/server.crt
keyfile C:/Program Files/mosquitto/server.key
```

Βήμα 5 — Εκκίνηση Broker στο port 8883

```
C:\Windows\System32>cd "C:\Program Files\mosquitto"

C:\Program Files\Mosquitto>mosquitto.exe -c mosquitto.conf -v
1773847882: mosquitto version 2.1.2 starting
1773847882: Config loaded from mosquitto.conf.
1773847882: Bridge support available.
1773847882: Persistence support available.
1773847882: TLS support available.
1773847882: TLS-PSK support available.
1773847882: Websockets support available.
1773847882: Plugin builtin-security has registered to receive 'basic-auth' events.
1773847882: Plugin builtin-security has registered to receive 'acl-check' events.
1773847882: Opening ipv6 listen socket on port 8883.
1773847882: Opening ipv4 listen socket on port 8883.
1773847882: mosquitto version 2.1.2 running
1773848004: New connection from ::1:63754 on port 8883.
1773848004: New client connected from ::1:63754 as subscriber-fdeizg (p4, c1, k60, u'subscriber_user').
1773848004: No will message specified.
1773848004: Client subscriber-fdeizg negotiated TLSv1.3 cipher TLS_AES_256_GCM_SHA384
1773848004: Sending CONNACK to subscriber-fdeizg (0, 0)
1773848004: Received SUBSCRIBE from subscriber-fdeizg
1773848004:     iot/temperature (QoS 0)
1773848004: subscriber-fdeizg 0 iot/temperature
```

Βήμα 6 — Αρχεία Publisher & Subscriber

mosquitto.conf | publisher.py | subscriber.py

```python
import random
import uuid
from datetime import datetime, UTC

BROKER = "localhost"
PORT = 8883
TOPIC = "iot/temperature"
CLIENT_ID = f"iot-publisher-{uuid.uuid4().hex[:8]}"
CA_CERT = r"C:\Users\conmy\Documents\project9\q3_tls\certs\ca.crt"

def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print(f"Publisher connected with CLIENT_ID: {CLIENT_ID}")
    else:
        print(f"Failed to connect, return code {reason_code}")

client = mqtt.Client(
    client_id=CLIENT_ID,
    callback_api_version=mqtt.CallbackAPIVersion.VERSION2
)
client.on_connect = on_connect
client.username_pw_set("publisher_user", "pub_pass123")
client.tls_set(ca_certs=CA_CERT)
client.tls_insecure_set(True)
client.connect(BROKER, PORT, keepalive=60)
client.loop_start()

try:
    while True:
        payload = {
            "sensor_id": "temp_sensor",
            "temperature_c": round(random.uniform(20.0, 30.0), 2),
            "timestamp": datetime.now(UTC).isoformat()
        }
        client.publish(TOPIC, json.dumps(payload))
        print(f"Published: {payload}")
        time.sleep(5)
except KeyboardInterrupt:
    print("Stop publisher!")
    client.loop_stop()
    client.disconnect()
```

mosquitto.conf | publisher.py | subscriber.py

```python
import paho.mqtt.client as mqtt
import random
import string

CA_CERT = r"C:\Users\conmy\Documents\project9\q3_tls\certs\ca.crt"

def generate_client_id(prefix):
    random_suffix = ''.join(random.choices(string.ascii_lowercase + string.digits, k=6))
    return f"{prefix}-{random_suffix}"

broker = "localhost"
port = 8883
topic = "iot/temperature"
client_id = generate_client_id("subscriber")
print(f"Subscriber ON - Client ID: {client_id}")

def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print("Connected to MQTT broker successfully")
        client.subscribe(topic, qos=0)
    else:
        print(f"Connection failed with code {reason_code}")

def on_message(client, userdata, msg):
    print(f"Received on {msg.topic}: {msg.payload.decode()}")

client = mqtt.Client(
    client_id=client_id,
    callback_api_version=mqtt.CallbackAPIVersion.VERSION2
)
client.on_connect = on_connect
client.on_message = on_message
client.username_pw_set("subscriber_user", "sub_pass123")
client.tls_set(ca_certs=CA_CERT)
client.tls_insecure_set(True)
client.connect(broker, port)
client.loop_forever()
```

Αποτελέσματα

Publisher στέλνει κρυπτογραφημένα δεδομένα:

Command Prompt - py q3_tls | Command Prompt - py q3_tls

```
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 24.4, 'timestamp': '2026-03-18T15:3
5:02.879385+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 28.34, 'timestamp': '2026-03-18T15:
35:07.880564+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 23.9, 'timestamp': '2026-03-18T15:3
5:12.881701+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 22.83, 'timestamp': '2026-03-18T15:
35:17.883515+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 20.47, 'timestamp': '2026-03-18T15:
35:22.884955+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 25.76, 'timestamp': '2026-03-18T15:
35:27.885910+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 27.53, 'timestamp': '2026-03-18T15:
35:32.886969+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 22.74, 'timestamp': '2026-03-18T15:
35:37.888247+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 23.18, 'timestamp': '2026-03-18T15:
35:42.889387+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 23.55, 'timestamp': '2026-03-18T15:
35:47.890440+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 22.33, 'timestamp': '2026-03-18T15:
35:52.891527+00:00'}
Published: {'sensor_id': 'temp_sensor', 'temperature_c': 23.01, 'timestamp': '2026-03-18T15:
35:57.892190+00:00'}
```

Subscriber λαμβάνει κρυπτογραφημένα δεδομένα:

```
Command Prompt - py q3_tls    Command Prompt - py q3_tls
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 28.97, "timestamp
": "2026-03-18T15:34:52.876340+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 28.07, "timestamp
": "2026-03-18T15:34:57.877691+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 24.4, "timestamp"
: "2026-03-18T15:35:02.879385+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 28.34, "timestamp
": "2026-03-18T15:35:07.880564+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 23.9, "timestamp"
: "2026-03-18T15:35:12.881701+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 22.83, "timestamp
": "2026-03-18T15:35:17.883515+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 20.47, "timestamp
": "2026-03-18T15:35:22.884955+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 25.76, "timestamp
": "2026-03-18T15:35:27.885910+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 27.53, "timestamp
": "2026-03-18T15:35:32.886969+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 22.74, "timestamp
": "2026-03-18T15:35:37.888247+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 23.18, "timestamp
": "2026-03-18T15:35:42.889387+00:00"}
Received on iot/temperature: {"sensor_id": "temp_sensor", "temperature_c": 23.55, "timestamp
": "2026-03-18T15:35:47.890440+00:00"}
```

Συμπέρασμα

Η επικοινωνία μεταξύ clients και broker κρυπτογραφείται πλέον μέσω TLS στο port 8883. Χρησιμοποιήθηκε self-signed CA certificate το οποίο εκδόθηκε με OpenSSL και το Common Name ορίστηκε ως localhost για τοπική χρήση.

Ερώτημα 4: Single Point of Failure & Αντιμετώπιση DDoS

Περιγραφή

Ένας MQTT broker εγκατεστημένος σε έναν μόνο server αποτελεί Single Point of Failure (SPOF) — αν ο server αποτύχει, ολόκληρο το IoT σύστημα παύει να λειτουργεί. Επιπλέον, είναι ευάλωτος σε επιθέσεις DDoS που στοχεύουν στην εξάντληση των πόρων του.

i. Αύξηση Διαθεσιμότητας

Clustering / Broker Redundancy Χρήση πολλαπλών MQTT brokers σε cluster, όπως το **EMQX** ή το **HiveMQ**, που υποστηρίζουν κατανεμημένη αρχιτεκτονική. Αν ένας broker αποτύχει, οι clients συνδέονται αυτόματα σε άλλον.

Load Balancer Τοποθέτηση Load Balancer (π.χ. HAProxy, NGINX) μπροστά από τους brokers για κατανομή του traffic. Παρέχει και health checking — αν ένας broker πέσει, ο Load Balancer σταματά να του στέλνει connections.

Message Persistence Ενεργοποίηση persistence στον Mosquitto (persistence true) ώστε τα μηνύματα να αποθηκεύονται στο δίσκο. Σε περίπτωση επανεκκίνησης δεν χάνονται δεδομένα.

Αυτόματη Επανεκκίνηση Ρύθμιση του broker ως system service με αυτόματη επανεκκίνηση σε περίπτωση αποτυχίας (RestartAlways σε Linux systemd ή Install Service σε Windows).

ii. Περιορισμός DDoS Επιθέσεων

Rate Limiting Connections Περιορισμός αριθμού συνδέσεων ανά IP στον Mosquitto με τις παραμέτρους:

```
max_connections 100
```

Firewall Rules Χρήση firewall (π.χ. iptables, Windows Defender Firewall) για περιορισμό πρόσβασης στα ports 1883/8883 μόνο από γνωστές IP διευθύνσεις.

Authentication & ACL Όπως υλοποιήθηκε στα Ερωτήματα 1 και 2 — η αυθεντικοποίηση αποτρέπει μαζικές ανώνυμες συνδέσεις που αποτελούν βασικό φορέα DDoS.

TLS Όπως υλοποιήθηκε στο Ερώτημα 3 — το TLS αυξάνει το υπολογιστικό κόστος για τον επιτιθέμενο, μειώνοντας την αποτελεσματικότητα της επίθεσης.

CDN / Anti-DDoS Service Χρήση υπηρεσιών όπως Cloudflare ή AWS Shield για απορρόφηση μεγάλου όγκου traffic πριν φτάσει στον broker.

Message Size Limiting Περιορισμός μεγέθους μηνυμάτων στον Mosquitto:

```
message_size_limit 1024
```

Συμπέρασμα

Η αντιμετώπιση του SPOF και των DDoS επιθέσεων απαιτεί συνδυασμό αρχιτεκτονικών λύσεων (clustering, load balancing) και τεχνικών μέτρων ασφάλειας (rate limiting, firewall, authentication). Τα μέτρα των Ερωτημάτων 1-3 αποτελούν τη βάση της θωράκισης, ενώ η κατανεμημένη αρχιτεκτονική εξασφαλίζει συνέχεια λειτουργίας σε περίπτωση αποτυχίας.